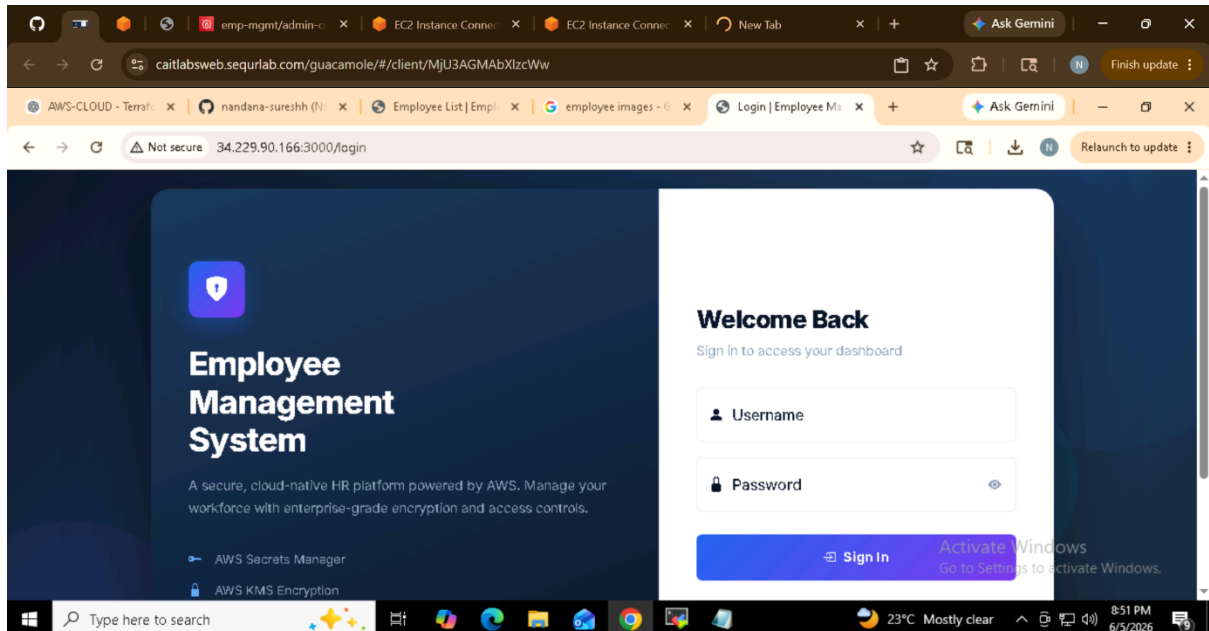
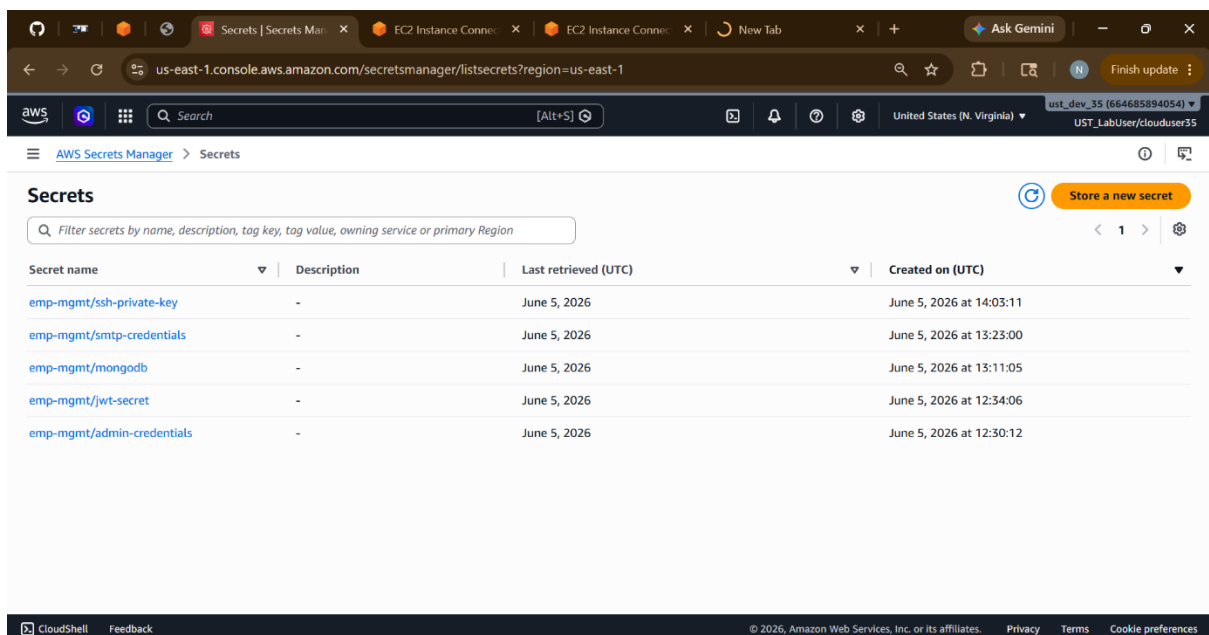


SECRETS MANAGER

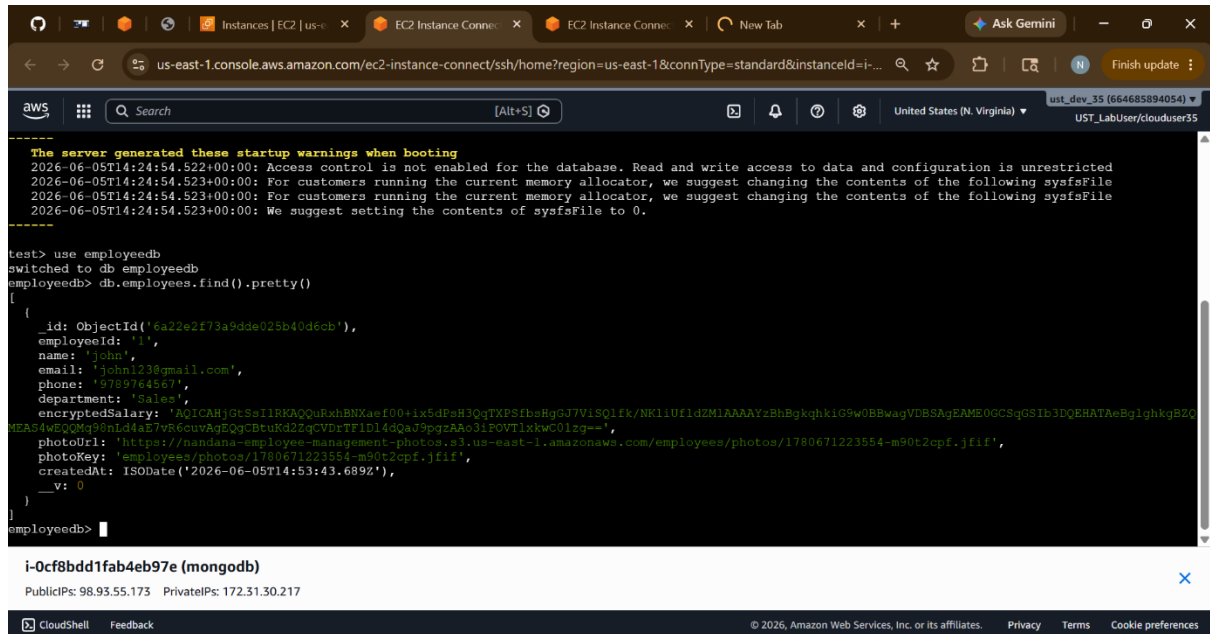
Developed a monolithic Employee Management System using Node.js, Express.js and MongoDB integrated with AWS services.



Implemented AWS Secrets Manager to securely store and retrieve MongoDB credentials, admin credentials, JWT secret, SMTP credentials. No credentials are hardcoded in the application.



Implemented AWS KMS to encrypt employee salary data before storing it in MongoDB. Verified that only encrypted ciphertext is stored in the database and decrypted only when viewing employee details.



The screenshot shows an AWS CloudShell terminal window. At the top, there are browser tabs for 'Instances | EC2 | us-...' and 'EC2 Instance Conne...'. The address bar shows 'us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-...'. The terminal output shows startup warnings from MongoDB, followed by a series of commands: 'test> use employeedb', 'switched to db employeedb', and 'employee> db.employee.find().pretty()'. The output is a JSON document for an employee named 'john' with fields for _id, employeeId, name, email, phone, department, encryptedSalary, photoUrl, photoKey, and createdAt. The encryptedSalary field contains a long base64-encoded string. Below the terminal output, the MongoDB instance details are shown: 'i-0cf8bdd1fab4eb97e (mongodb)' with PublicIPs: 98.93.55.173 and PrivateIPs: 172.31.30.217. The bottom of the terminal shows 'CloudShell Feedback' and copyright information for Amazon Web Services.

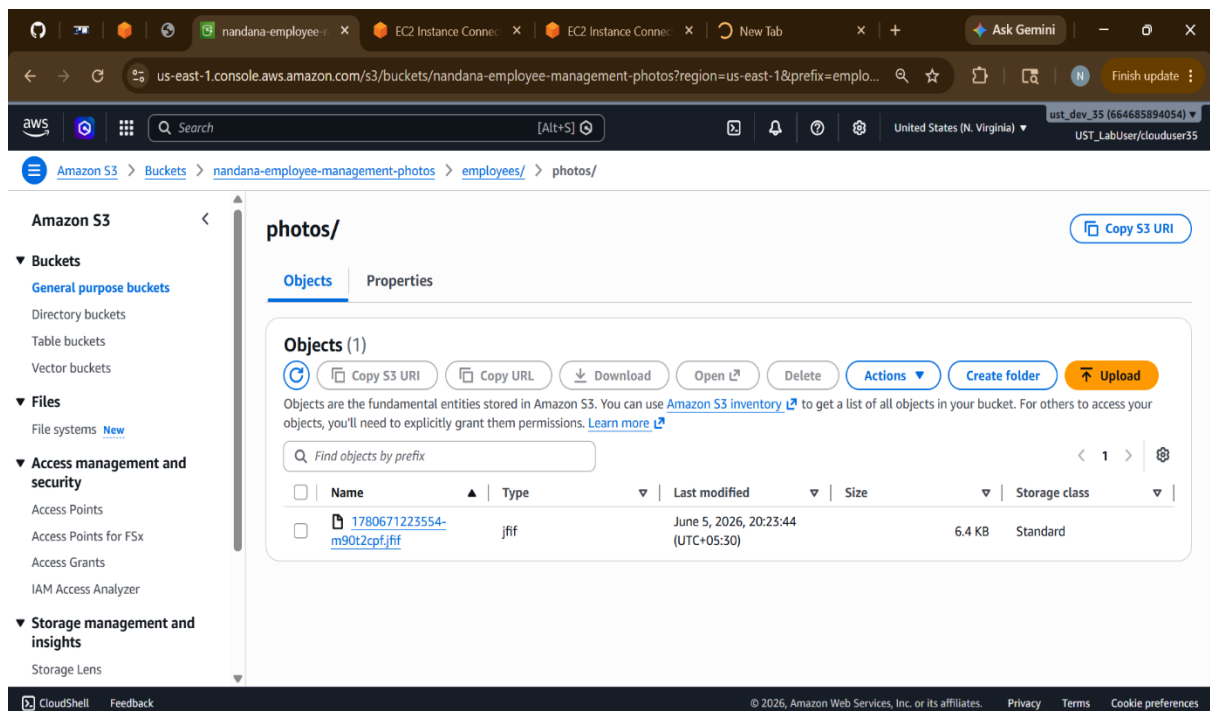
```
-----
The server generated these startup warnings when booting
2026-06-05T14:24:54.522+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-06-05T14:24:54.523+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-06-05T14:24:54.523+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2026-06-05T14:24:54.523+00:00: We suggest setting the contents of sysfsFile to 0.
-----

test> use employeedb
switched to db employeedb
employee> db.employee.find().pretty()
[
  {
    _id: ObjectId('6a22e2f73a9d3e025b40d6cb'),
    employeeId: '1',
    name: 'john',
    email: 'john123@gmail.com',
    phone: '9789764567',
    department: 'Sales',
    encryptedSalary: 'AQICAHjGtSaIIRKAQQuRxbBNXaeF00+ix5dPsH3QqTXPSfbsHgGJ7ViSQ1fk/NK1iUfIdZM1AAAYzBhBgkqhkiG9w0BBwagYDZSAgEAME0GCSqGSIb3DQEHAATAgB1ghkgBQ0CMEAS4wEQMq9bnld4aE/vR6cuvAgEQGCBtuKd22qGVdtTFID143QaJ9pgzAAo31POVTlkkwCUIzg=',
    photoUrl: 'https://nandana-employee-management-photos.s3.us-east-1.amazonaws.com/employees/photos/1780671223554-m90t2cpf.jfif',
    photoKey: 'employees/photos/1780671223554-m90t2cpf.jfif',
    createdAt: ISODate('2026-06-05T14:53:43.689Z'),
    __v: 0
  }
]
employee>
```

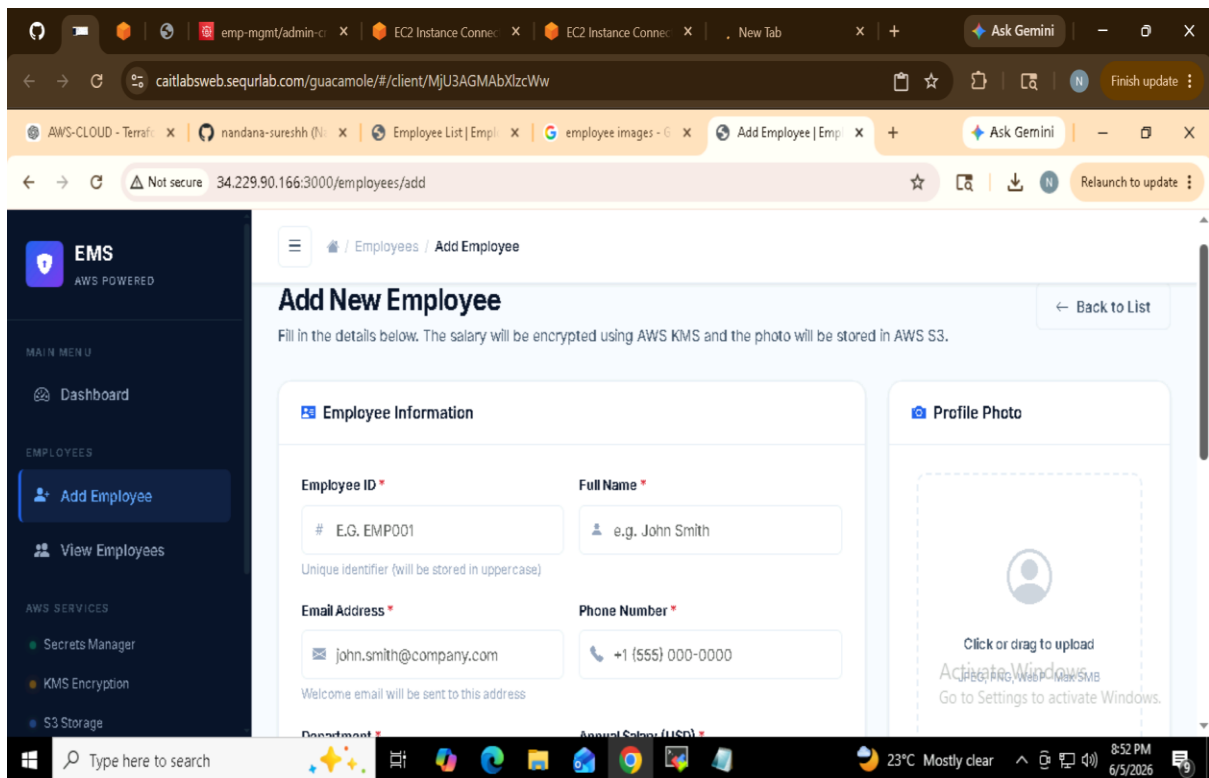
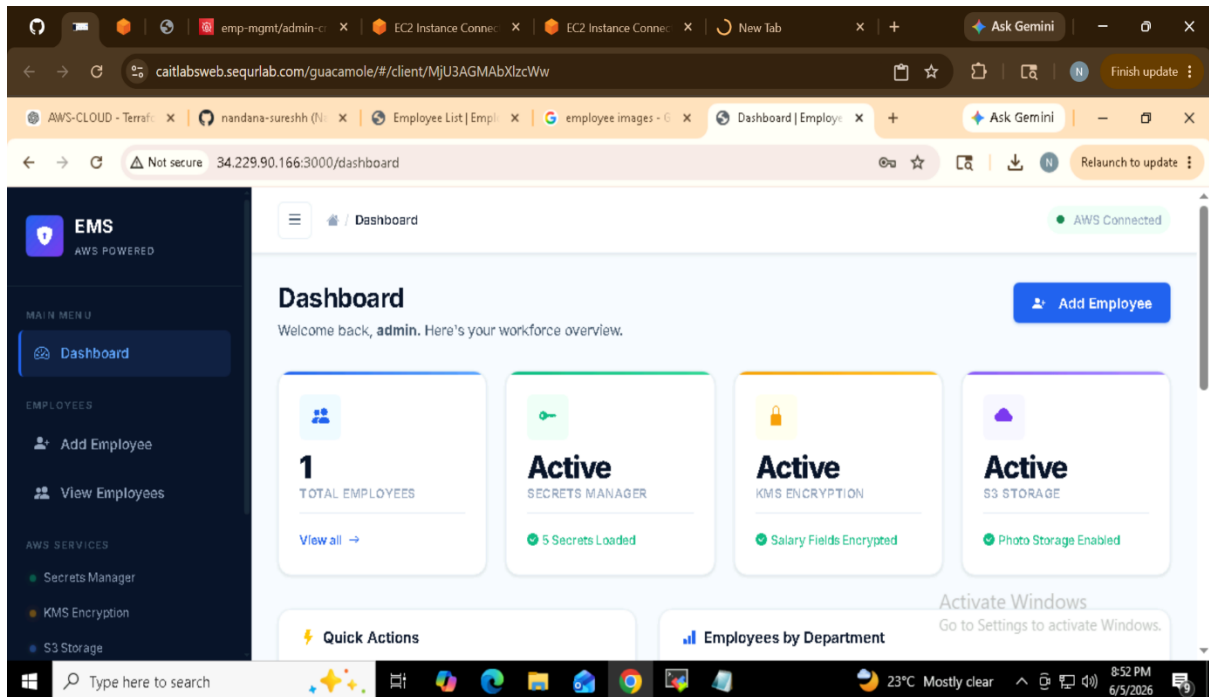
i-0cf8bdd1fab4eb97e (mongodb)
PublicIPs: 98.93.55.173 PrivateIPs: 172.31.30.217

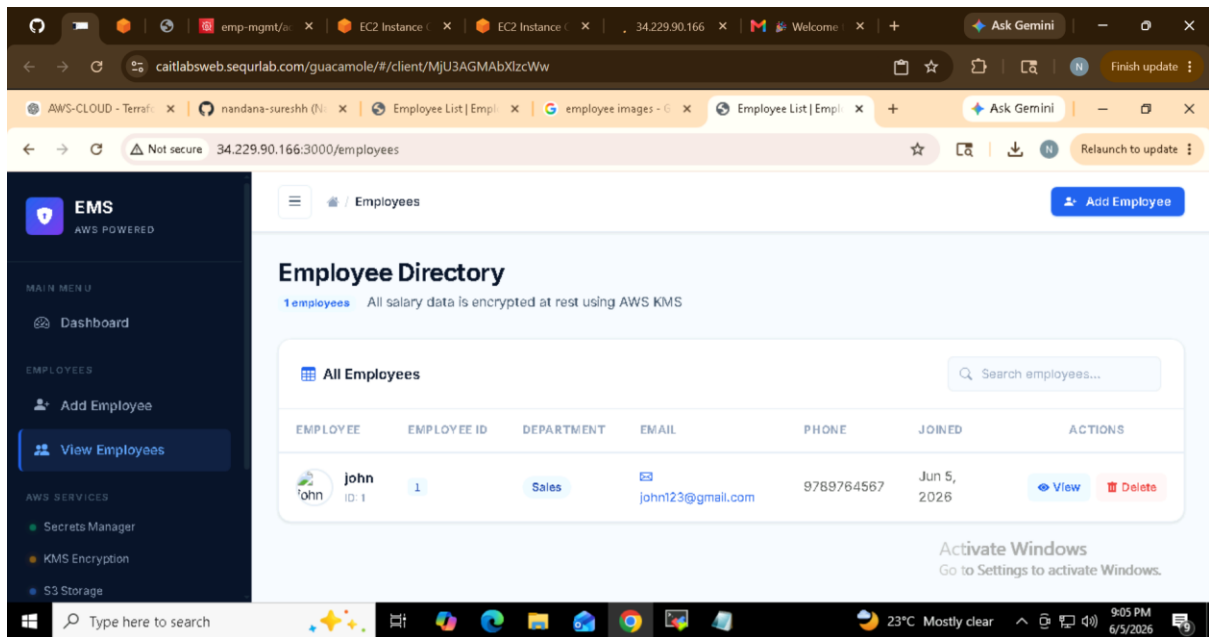
CloudShell Feedback © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Implemented AWS S3 for employee profile photo storage. Employee images are uploaded to S3 and the object URL is stored in MongoDB.

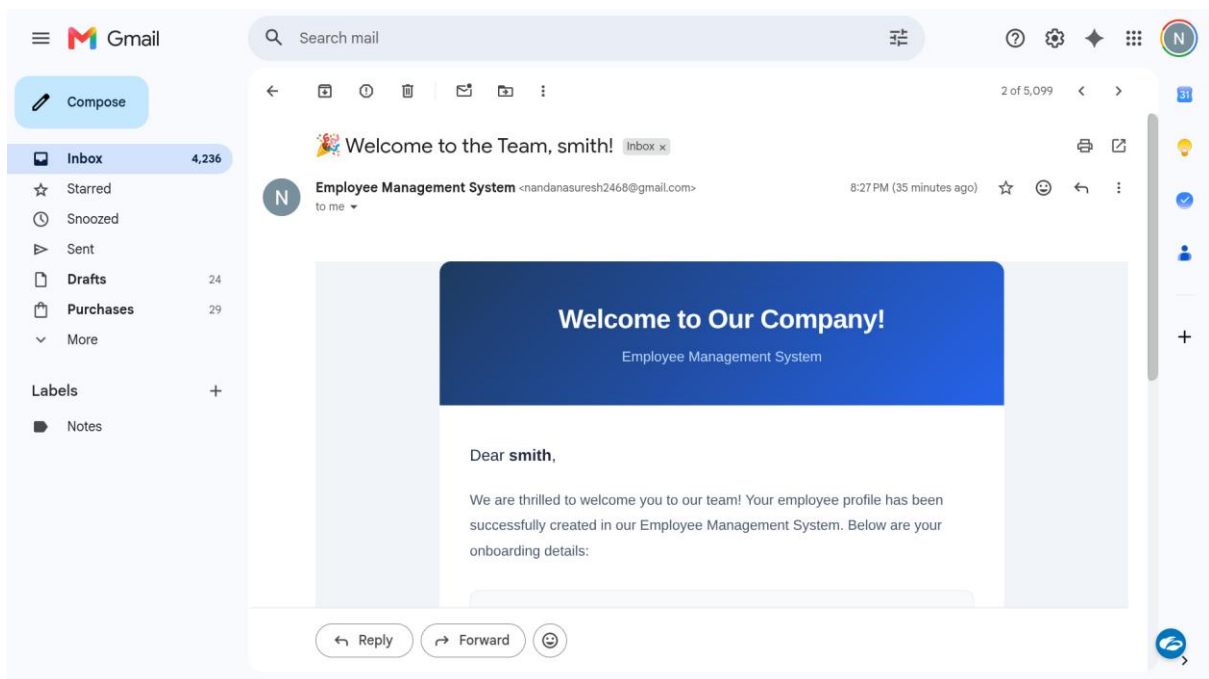


Implemented JWT-based authentication using credentials retrieved from AWS Secrets Manager.





Configured SMTP email integration using credentials stored in Secrets Manager to send automated welcome emails when employees are created.



Deployed the application on EC2 and connected it to a MongoDB instance running on a separate EC2 server. Verified end-to-end functionality including login, employee creation, KMS encryption/decryption, S3 uploads, MongoDB storage, and email notifications.